SAP S/4HANA UND SAP ERP · ON-PREMISE · KLASSISCHES ABAP

Documizer

Dokumentation, Transfer und KI-Analyse für SAP-Entwicklungen und Customizing

Ein einziger ABAP-Report, in Sekunden installiert. Er dokumentiert, was Ihr System **wirklich** enthält — und bringt es auf Wunsch in ein anderes System. Ohne Transport, ohne abapGit, ohne zusätzliche Repository-Objekte.

3	1	20+	0
Reports: Customizing, Entwicklung, Prüfkonfiguration	Quelltext je Report — per Copy & Paste in SE38	Objektarten, die sich übertragen lassen	Transporte, Installationen oder Zusatzobjekte

Produktdokumentation · Stand Oktober 2026

Christian Gathmann · gathmann@csbg.de
C. Gathmann Computer-System-Beratung GmbH (CSBG), Bremen

Auf einen Blick

Wer ein SAP-System verantwortet, kennt das Problem: Die Dokumentation beschreibt, was einmal geplant war. Was tatsächlich im System steht, weiß nur das System selbst. Der Documizer fragt es — und macht aus der Antwort ein Dokument, das Menschen lesen, Systeme wieder einlesen und KI-Assistenten auswerten können.

`ZDOCUMIZER_IMG`

Customizing

Liest einen Strukturbaum des Einführungsleitfadens, übernimmt SAPs Dokumentation je IMG-Aktivität und liest das Customizing dahinter bis in die Tabelleninhalte. Erzeugt auf Wunsch ein BC-Set.

`ZDOCUMIZER_DEV`

Entwicklungen

Startet an einem Paket, einem einzelnen Objekt oder einem Transportauftrag — Report, Klasse, Tabelle, CDS, RAP, IDoc, UI5-App, Formular. Liefert Felder, Methoden, Texte, Quelltext, Verwendungen. Und importiert.

`ZDOCUMIZER_ATC`

Prüfkonfiguration

Dokumentiert ATC- und Code-Inspector-Varianten: welche Checks laufen, mit welcher Priorität, Kundenpriorisierung, Pseudokommentar, Pragma und Ausnahmen.

Was den Documizer auszeichnet

- **Installiert in Sekunden.** Jeder Report ist eine einzige Quelle. Einfügen in SE38, aktivieren, fertig — auch dort, wo nichts installiert und kein Transport eingespielt werden darf.
- **Ein Format für alles.** Das XML ist vollständig, versioniert und lässt sich verlustfrei wieder einlesen. Das HTML ist in sich geschlossen und öffnet sich auch in Jahren noch ohne Nacharbeit.
- **Dokumentation, die zurück ins System kann.** Der Development-Report legt aus demselben Dokument Tabellen, Klassen, CDS-Views, RAP-Objekte, IDoc-Typen, UI5-Apps und Adobe-Formulare in einem anderen System an.
- **Auch ein Transport ist eine Quelle.** Auftragsnummer eintragen, und das Dokument beschreibt genau das, was dieser Transport enthält — mit Quelltext und Verwendungen.
- **Nicht nur S/4HANA.** Dieselbe Quelle läuft im alten ERP — geprüft bis hinunter zu SAP_BASIS 7.40. Was es dort nicht gibt, sucht der Report dynamisch und lässt es leer, statt abzubrechen.
- **Nie stillschweigend unvollständig.** Greift eine Mengengrenze, steht das im Dokument. Jeder Lauf kann sich selbst prüfen: serialisieren, einlesen, vergleichen.
- **Sicher im Kundensystem.** Die Berechtigungsprüfungen von SAP bleiben eingeschaltet. Customizing wird nie direkt geschrieben, sondern als BC-Set zur Prüfung angeboten.
- **Offen.** Apache-2.0-Lizenz, Quellen auf GitHub — siehe Kapitel 12.

Inhalt

Auf einen Blick 2
1 Warum Documizer 4
2 Sieben Einsatzszenarien 5
3 Installation in Sekunden 8
4 Customizing dokumentieren: ZDOCUMIZER_IMG 9
5 Customizing übertragen: BC-Sets und Fehlliste 11
6 Entwicklungen dokumentieren: ZDOCUMIZER_DEV 12
7 Prüfstandards dokumentieren: ZDOCUMIZER_ATC 16
8 Ausgabe: XML und HTML 17
9 Import: Entwicklungen in ein anderes System bringen 18
10 Sicherheit und Betrieb 21
11 Grenzen und Ausblick 22
12 Bezug, Lizenz und Mitwirkung 23
13 Kontakt 24

1 Warum Documizer

SAP-Systeme wachsen über Jahre. Pakete, Klassen, CDS-Views, Customizing-Einträge — und mit jedem Release driftet die Dokumentation weiter von der Wirklichkeit weg.

Vier Probleme aus dem Projektalltag

Problem	Was es kostet
Dokumentation veraltet. Sie beschreibt den Entwurf, nicht den Stand im System.	Einarbeitung dauert Wochen; Fehler werden an der falschen Stelle gesucht.
Kundensysteme sind abgeschottet. Keine Installation, keine Transporte, abapGit ist keine Option.	Werkzeuge, die im eigenen System selbstverständlich sind, stehen beim Kunden nicht zur Verfügung.
Wissen steckt im System. Wer was verwendet, welche Felder gelesen werden, welche Checks greifen — das steht in Tabellen, nicht in Dokumenten.	Auswirkungsanalysen werden geschätzt statt nachgelesen.
KI braucht Kontext. Ein Assistent hilft nur so gut, wie die Informationen, die er bekommt.	Einzelne Code-Schnipsel reichen für Recherche und Fehlersuche selten aus.

Die Antwort

Der Documizer liest direkt aus dem laufenden System — über die Standard-APIs von SAP, wo es sie gibt. Er schreibt dabei nichts, außer wenn Sie ihn ausdrücklich zum Import auffordern. Das Ergebnis ist ein kapitelweise gegliedertes Dokument in zwei Formaten:

- **XML** — vollständig, versioniert, maschinenlesbar. Das Transportformat für den Import und die ideale Eingabe für KI-Assistenten.
- **HTML** — eine einzige Datei mit Inhaltsverzeichnis, eingebettetem Stil und ohne JavaScript. Zum Lesen, Weitergeben, Ablegen.

> **Das Prinzip**
> Eine Quelle je Report. Ein Format für Mensch, System und KI. Keine Installation im Zielsystem.

2 Sieben Einsatzszenarien

Derselbe Report, dieselbe Datei — sieben Aufgaben, für die sonst jeweils ein eigenes Werkzeug nötig wäre.

Entwicklungen dokumentieren

DIE AUFGABE

Ein Paket, eine Anwendung oder ein einzelnes Objekt soll vollständig beschrieben werden — für Übergabe, Wartung, Audit oder Einarbeitung neuer Kollegen.

WAS DER DOCUMIZER LIEFERT

- Kurztexte, Autor und Paket je Objekt, auf Wunsch die Objektdokumentation
- Feldlisten von Tabellen, Strukturen, Views und CDS-Views mit Datenelementen und Schlüsseln
- Klassen mit Oberklasse, Schnittstellen, Komponenten und lokalen Klassen — auch die Handler eines RAP-Behavior-Pools
- Quelltext, Verwendungsnachweis mit Fundstellen und die Objekte, die verwendet werden

> **Ihr Nutzen**
> Eine Dokumentation, die dem System entspricht — auf Knopfdruck neu erzeugt, statt mühsam nachgepflegt.

Einen Transport dokumentieren

DIE AUFGABE

Was steckt in diesem Auftrag? Vor einer Abnahme, einem Review oder einem Import in die Produktion soll nachlesbar sein, was er tatsächlich verändert.

WAS DER DOCUMIZER LIEFERT

- Auftragsnummer als Startpunkt — die Wertehilfe zeigt die eigenen Aufträge
- Kopf mit Art, Status, Kurztext, Eigentümer, Datum, Zielsystem und den Aufgaben darunter
- Jedes Objekt des Auftrags als Kapitel, mit Details, Quelltext und Verwendungen
- Customizing-Einträge, Löschungen und fehlende Objekte werden vorweg benannt statt übergangen

> **Ihr Nutzen**
> Ein Transport wird lesbar, bevor er ankommt — und mit dem Testlauf des Imports sehen Sie sogar, was er im Zielsystem verändern würde.

IMG-Einstellungen dokumentieren

DIE AUFGABE

Das Customizing eines Bereichs oder Projekts soll nachvollziehbar festgehalten werden — mit Erklärung und tatsächlichen Werten.

WAS DER DOCUMIZER LIEFERT

- Strukturbaum des Einführungsleitfadens mit allen IMG-Aktivitäten
- SAPs Standarddokumentation je Aktivität, als lesbarer Text aufbereitet
- Die Customizing-Tabellen dahinter mit Feldliste und Inhalt — auf Wunsch nur die kundeneigenen Einträge
- Projektstatus und Memo aus einem IMG-Projekt

> **Ihr Nutzen**
> Die Customizing-Dokumentation entsteht aus dem System selbst. SAPs Erklärung steht direkt neben Ihren Werten.

Entwicklungen übertragen

DIE AUFGABE

Objekte sollen in ein anderes System — etwa aus der Sandbox ins Kundensystem —, wo weder Transporte eingespielt noch abapGit installiert werden können.

WAS DER DOCUMIZER LIEFERT

- Definitionsdaten im XML: dieselben Strukturen, die SAP beim Lesen liefert und beim Anlegen erwartet
- Import von über 20 Objektarten, von der Domäne bis zum Adobe-Formular
- Testlauf als Vorbelegung, bestehende Objekte bleiben unangetastet
- Objekte einzeln abwählbar oder umbenennbar, Paketregeln alt → neu, Auftrag und Aufgabe automatisch zugeordnet

> **Ihr Nutzen**
> Entwicklungen wandern als eine Datei. Das Zielsystem braucht nur den Documizer selbst.

Customizing übertragen per BC-Set

DIE AUFGABE

Customizing aus einem System soll in einem anderen nachgezogen werden — kontrolliert und nachvollziehbar.

WAS DER DOCUMIZER LIEFERT

- Aus dem IMG-Dokument entsteht ein Business Configuration Set
- Aktiviert wird in SCPR20: mit Protokoll, Transportauftrag, Vorschau und Weg zurück
- Eine Fehlliste nennt die Tabellen, die im Zielsystem noch fehlen — der Development-Report exportiert genau diese

> **Ihr Nutzen**
> Kein Report schreibt in Ihre Customizing-Tabellen. Sie entscheiden, was aktiviert wird.

KI-gestützte Analyse

DIE AUFGABE

Ein KI-Assistent wie Claude soll bei Recherche, Fehlersuche oder Dokumentation helfen — und braucht dafür den vollständigen Zusammenhang.

WAS DER DOCUMIZER LIEFERT

- Ein strukturiertes XML mit Quelltexten, Feldlisten, Verwendungen und Customizing in einem Stück
- Eindeutige Kapitel und Objektreferenzen, auf die sich Fragen beziehen lassen
- Vermerke, wo etwas gekürzt wurde — der Assistent weiß, was er nicht sieht

> **Ihr Nutzen**
> Statt einzelne Schnipsel zu kopieren, geben Sie dem Assistenten die ganze Anwendung — und bekommen fundierte Antworten.

Prüfstandards transparent machen

DIE AUFGABE

Welche ATC-Checks laufen, mit welcher Priorität, und wie lassen sich Meldungen unterdrücken? Die Antwort steht verteilt in Clustern und Klassen.

WAS DER DOCUMIZER LIEFERT

- Prüfvarianten mit Kategorien, Checks und allen Meldungen
- Priorität der Checkklasse neben der kundeneigenen Priorisierung
- Pseudokommentar und Pragma je Meldung, erteilte Ausnahmen

> **Ihr Nutzen**
> Code-Standards werden nachlesbar und zwischen Systemen vergleichbar.

3 Installation in Sekunden

Jeder Report ist eine einzige Quelltextdatei. Es gibt nichts zu transportieren und nichts zu konfigurieren.

1. SE38 aufrufen, Programmnamen vergeben (etwa `ZDOCUMIZER_DEV`), **Anlegen** wählen, Typ *Ausführbares Programm*.
2. Quelltext einfügen, sichern, aktivieren.
3. Programmtitel unter *Springen* → *Attribute* eintragen — das einzige, was von Hand gesetzt wird.

> **Warum das funktioniert**
> Keine globalen Klassen, keine Textelemente, keine eigenen Tabellen, keine Simple Transformation, keine Nachrichtenklasse. Alle Bildschirmtexte stehen im Quelltext — auf Englisch, bei deutscher Anmeldung auf Deutsch.
>
> XML wird über die Kernelklassen `cl_sxml_string_writer` und `cl_sxml_string_reader` geschrieben und gelesen. Sie sind in jedem System vorhanden.

Reportdokumentation im System (optional)

Der Hilfsreport `ZDOCUMIZER_DOCU` schreibt die SAP-Reportdokumentation für die drei Reports — sichtbar in SE38 unter *Dokumentation* und über das Info-Symbol des Selektionsbilds. Für die Funktion wird er nicht gebraucht; sinnvoll ist er, wenn im System weitere Kollegen mit dem Documizer arbeiten.

Systemvoraussetzungen

Punkt	Anforderung
Plattform	SAP S/4HANA On-Premise **und** SAP ERP (R/3), klassisches ABAP
Release	Verifiziert auf SAP_BASIS 758 (S/4HANA 2023). Für ältere Stände umgebaut: POSIX-`REGEX` statt `PCRE`, keine Konstruktoren, die erst ab 7.55 gelten — der IMG-Report läuft nachweislich auf einem Kundensystem mit SAP_BASIS 7.40
Berechtigung Dokumentation	Lesender Zugriff auf das Repository; für Customizing-Inhalte `S_TABU_NAM`
Berechtigung Import	`S_DEVELOP` für die anzulegenden Objekte und Pakete — die SAP-Prüfungen bleiben eingeschaltet
Betrieb	Dialog und Hintergrund; im Hintergrund Ablage auf dem Applikationsserver

> **Auch im alten ERP**
> Neuere Objektarten gibt es in einem älteren System nicht — RAP, CDS, UI5. Der Report sucht sie deshalb dynamisch und läuft auch dort, wo es sie nicht gibt: was fehlt, bleibt leer, statt den Lauf abzubrechen.

4 Customizing dokumentieren: ZDOCUMIZER_IMG

Der IMG-Report liest einen Strukturbaum des Einführungsleitfadens, ordnet jedem Knoten seine IMG-Aktivität zu und geht bis zu den Customizing-Einträgen hinunter.

Was im Dokument steht

```
1  <Wurzelknoten>                        Gliederungskapitel
   1.1 <Unterknoten>                     Gliederungskapitel
       1.1.1 <IMG-Aktivität>             Aktivitätskapitel
             ├─ Beschreibung             SAP-Dokumentation, lesbar aufbereitet
             ├─ Technische Zuordnung     Aktivität, Transaktion, Dokuobjekt
             ├─ Projektstatus / Memo     im Projektmodus
             ├─ Objekt je Zuordnung      Feldliste, Auswahlverfahren, Zeilenzahl
             ├─ Customizing-Inhalt       Daten gemäß Mengensteuerung
             ├─ Transport-Historie       Aufträge zur Tabelle
             └─ Verwendete Objekte       Programm hinter der Transaktion
```

Selektionsbild

Feld	Vorbelegung	Wirkung
Struktur-ID (GUID) / oder Suchbegriff Wurzel		Einstieg. Die eigene Wertehilfe zeigt Wurzelknoten mit Text, Knotenzahl und Änderungsdatum — statt 500 nackter GUIDs
Projekt (Status/Memo) / nur Projektknoten		Überlagert den Baum mit einem IMG-Projekt, beschneidet auf dessen Knoten
Max. Tiefe / Max. Kapitel	0 / 300	Umfang des Baums; 0 = alle
Unterstrukturen auflösen	an	Entscheidend: ohne dies endet der Lauf nach der ersten Ebene
Standarddokumentation / Objektzuordnung	an / an	SAP-Doku je Aktivität, zugeordnete Customizing-Objekte
Customizing-Inhalte lesen	an	Feldliste und Tabelleninhalte
nur kundeneigene Einträge / Max. Zeilen	an / 500	Mengensteuerung, siehe unten
Transport-Historie / Ausgehende Aufrufe	aus / aus	Aufträge je Tabelle; Programm hinter der Transaktion

Nur, was Ihnen gehört

Customizing-Tabellen reichen von drei Zeilen bis zu Millionen. Mit *nur kundeneigene Einträge* wirken zwei Verfahren zugleich: der **Namensraum** des ersten Schlüsselfelds (Y, Z, /) und das **Vorkommen in Transportaufträgen** zu dieser Tabelle. So werden auch nie transportierte Einträge und Schlüssel außerhalb des Kundennamensraums gefunden.

Welches Verfahren gegriffen hat, steht am Objekt — nicht nur im Protokoll. Fehlt die Leseberechtigung, erscheint das Objekt mit Vermerk statt stillschweigend leer.

Projektmodus

Ein IMG-Projekt trägt je Aktivität Status, Fortschritt und Memo. Der Report überlagert den Baum damit und kann auf die bearbeiteten Knoten beschneiden. Gliederungskapitel bleiben erhalten, solange darunter ein Projektknoten hängt; die Kapitelnummern behalten ihre Position, Lücken zeigen, was ausgelassen wurde.

> **Gemessen**
> Projektmodus mit 351 Kapiteln: **0,48 Sekunden**. Der Einstieg wird aus den Projektdaten abgeleitet — statt über den ganzen Leitfaden zu laufen.

5 Customizing übertragen: BC-Sets und Fehlliste

Die Gegenrichtung des IMG-Reports: aus einem Dokument entsteht im Zielsystem ein Business Configuration Set. Geschrieben wird damit noch nichts.

Warum ein BC-Set und kein Direktzugriff

Das Customizing entsteht erst, wenn jemand das Set in **SCPR20** aktiviert. Aktivierung läuft mit Protokoll, mit Transportauftrag, mit Vorschau und mit einem Weg zurück. Ein Report, der direkt in Customizing-Tabellen schreibt, hätte davon nichts.

Sperre	Grund
Tabelle im Zielsystem unbekannt	Ein Set auf eine fehlende Tabelle lässt sich nicht aktivieren
Auslieferungsklasse nicht C, G oder E	Alles andere ist keine Customizing-Tabelle
Inhalt beim Export gekürzt	Ein gekürzter Inhalt sähe im Set vollständig aus — beim Aktivieren verschwände der Rest

Die Fehlliste schließt den Kreis

Kundeneigene Customizing-Tabellen gibt es im Zielsystem oft noch nicht — sie sind Entwicklung. Der IMG-Report schreibt deshalb eine **Fehlliste** der übergangenen Objekte. Im Quellsystem liest der Development-Report diese Liste als Startpunkt und exportiert genau diese Objekte. Nach deren Import läuft die BC-Set-Erzeugung auch für sie durch.

```
Zielsystem     IMG-Report       BC-Set erzeugen  →  Fehlliste: Z_SKIP_IND, ZV_SKIP_IND
Quellsystem    DEV-Report       Startpunkt AL    →  Export mit Definitionsdaten
Zielsystem     DEV-Report       Import           →  Tabelle und View angelegt
Zielsystem     IMG-Report       BC-Set erzeugen  →  vollständig
```

> **Ihr Nutzen**
> Das Zielsystem sagt selbst, was fehlt. Angefordert wird nur, was dort wirklich nicht vorhanden ist.

6 Entwicklungen dokumentieren: ZDOCUMIZER_DEV

Der Einstieg ist ein Paar aus Typ und Name, wie im Object Navigator. Ein Paket ergibt Paket → Objekttyp → Objekt; ein Einzelobjekt ergibt das Objekt, alles was es verwendet, und alle, die es verwenden.

Startpunkte

Code	Startpunkt	Code	Startpunkt
`K`	Paket (Vorbelegung)	`SB`	Service-Binding
`P`	Programm / Report	`IS`	IDoc-Segmenttyp
`C`	Klasse oder Interface	`IT`	IDoc-Basistyp
`FF`	Funktionsbaustein	`U5`	UI5-Anwendung
`F`	Funktionsgruppe	`SF`	Smart Form
`T`	Tabelle / Struktur	`AF`	Adobe Form (Formular)
`V`	View	`AI`	Adobe Form (Schnittstelle)
`D`	Datenelement	`TR`	**Transportauftrag / Aufgabe**
`DO`	Domäne	`AL`	Arbeitsliste aus Fehlliste
`W`	CDS View	`MX`	Metadata Extension
`BD`	Verhaltensdefinition	`SD`	Service-Definition

Zwei Codes benennen kein Objekt, sondern eine Liste davon: `AL` die Fehlliste aus dem Zielsystem und `TR` einen Transportauftrag. Jedes Objekt der Liste wird ein eigenes Kapitel.

Einen Transportauftrag dokumentieren

Mit `TR` steht im Namensfeld eine Auftrags- oder Aufgabennummer; die Wertehilfe zeigt die eigenen Aufträge, die neuesten zuerst. **Der Auftrag selbst steht in Kapitel 1** — mit Nummer, Art, Status, Kurztext, Eigentümer, Datum, Zielsystem und der Zahl der Aufgaben; die Objekte beginnen bei 2. Drei Eigenheiten des Korrektursystems nimmt der Report Ihnen ab:

- **Die Objekte hängen an den Aufgaben, nicht am Auftrag.** Die Objektliste eines Auftrags ist regelmäßig leer — transportiert wird, was in den Aufgaben darunter steht. Gelesen werden beide Ebenen.
- **Teile werden auf ihr Objekt zurückgeführt.** Ein Auftrag führt neben ganzen Objekten auch einzelne Methoden, Includes und Tabellendefinitionen auf. Dokumentiert wird das Objekt, zu dem der Teil gehört, und jedes nur einmal.
- **Was kein dokumentierbares Objekt ist, wird vorweg genannt:** Customizing-Einträge mit Tabelle und Zahl der Schlüssel samt Verweis auf den IMG-Report, Löschungen, Einträge ohne Katalogobjekt und Objekte, die es im System nicht mehr gibt.

> **Freigegeben oder änderbar**
> Ein freigegebener Auftrag dokumentiert, was drin *war*; ein änderbarer, was gerade drin *ist*. Im zweiten Fall steht das als Hinweis im Kopf — sonst liest sich beides gleich.

Was sich gegenüber dem vorigen Transport geändert hat

Mit dem Kennzeichen *Änderungen zur Vorversion* steht in jedem Objektkapitel, was dieser Transport gegenüber dem vorigen ändert — grün, rot und mit Umgebung, genau wie der Vergleich im Testlauf des Imports:

```
Aenderungen gegenueber EBSK900017 vom 2017-10-07: +4 / -1 Zeilen
     8   PARAMETERS: p_vbeln TYPE vbak-vbeln .
     9 - PARAMETERS : ordtype TYPE auart DEFAULT 'ZCM'.
     9 + *{   REPLACE        S4HK901040
    11 + PARAMETERS : ordtype TYPE auart DEFAULT 'OR'.
    13   PARAMETERS : fromdate TYPE dats,
```

Beide Fassungen kommen aus der **Versionsverwaltung**: Beim Freigeben legt SAP je Objekt eine Version an und merkt sich die Auftragsnummer dazu. Die Version dieses Transports ist die mit seiner Nummer, der Vorgänger die nächstältere — gleich aus welchem Auftrag sie stammt, auch aus einem anderen System.

- **Verglichen wird Teil für Teil.** Eine Klasse besteht in der Versionsverwaltung aus Definition, den drei Abschnitten und je Methode einem Eintrag, eine Funktionsgruppe aus ihren Bausteinen. Jeder Teil bekommt seine eigene Überschrift.
- **Noch nicht freigegeben?** Dann gibt es für den Auftrag noch keine Version, und es steht der aktuelle Stand gegen die jüngste vorhandene — das, was dieser Transport bringen *wird*, gegen das, was zuletzt transportiert wurde. Vor einer Abnahme ist genau das die Frage.
- **Nicht jede Objektart hat Versionen** — UI5-Anwendungen und Formulare etwa nicht. Dort entfällt der Vergleich; wie viele Objekte verglichen wurden, steht am Ende des Laufs.

Selektionsbild

Feld	Vorbelegung	Wirkung
Unterpakete rekursiv / lokale Pakete einbeziehen	an / aus	Umfang im Paketmodus
Objekttyp / Objektname		Filter auf die Objektliste
Max. Objekte	2000	Die Gesamtzahl bleibt im Dokument vermerkt
Objektdokumentation / Details	an / an	Doku; Felder, Methoden, Textelemente, Festwerte …
Quelltext ausgeben / Max. Zeilen je Objekt	an / 30000	Quelltext; eine Kürzung steht im Kopf des Dokuments
Verwendungsnachweis / auch für verwendete Objekte	an / an	Wer nutzt das Objekt — mit Fundstellen und Feldern
Ausgehende Aufrufe / Verweistiefe	an / 3	Was das Objekt nutzt, und wie tief dem gefolgt wird
Verwendungstiefe	3	Gefundene Verwender selbst als Kapitel dokumentieren

Feld	Vorbelegung	Wirkung
SAP-Standard ausschließen	an	Nur Eigenentwicklung in den Verweislisten
Definitionsdaten für Reimport	aus	Technische Definition für den Import — zieht die XML-Ausgabe nach sich

Details je Objektart

Objektart	Im Dokument
Programm	Titel, Selektionstexte, Textsymbole, Überschriften, Objektdokumentation
Klasse, Interface	Oberklasse, Schnittstellen, Komponenten, implementierte Schnittstellenmethoden, lokale Klassen mit Methoden; Quelltext als eine geschlossene Quelle
Funktionsbaustein, -gruppe	Schnittstelle mit allen Parametern; Bausteinliste
Tabelle, Struktur	Feldliste mit Datenelement oder eingebautem Typ und Schlüssel
View	Feldliste, Basistabellen mit Join-Bedingung
CDS View	Feldliste der Entität, verwendete Objekte, Quelltext
Verhaltensdefinition, Service-Definition, Metadata Extension	Beschreibung und Quelltext
Service-Binding	Bindungsart, verknüpfte Service-Definition, Publikationsstand
IDoc-Basistyp	Segmenthierarchie (WE60), Nachrichtentypen (WE82), verarbeitende Bausteine (WE57)
IDoc-Segment	Felder mit den Datenelementen, die WE31 zeigt
UI5-Anwendung	Beschreibung; mit Definitionsdaten alle Dateien samt Verzeichnisbaum
Smart Form	Knotenbaum, Schnittstelle, globale Definitionen und das Coding der Code-Knoten — gelesen aus dem Objektmodell des Form Builders
Adobe Form	Formular und seine Schnittstelle als Einheit: Kontext, Felder, Handler-Klasse; mit Definitionsdaten das Layout als Ganzes
Datenelement, Domäne	Feldbezeichner; Festwerte

Verweise in beide Richtungen

Ausgehende Aufrufe zeigen, was ein Objekt verwendet. Weil die Querverweisindizes von SAP jeweils nur einen Teil kennen, liest der Report zusätzlich den Quelltext: `CALL FUNCTION`, `SUBMIT`, Tabellenzugriffe, Includes, Nachrichtenklassen — und sogar Objektnamen in Zeichenliteralen, die zur Laufzeit gebraucht werden. Jeder Treffer wird gegen den Objektkatalog geprüft, bevor er ins Dokument kommt.

Der **Verwendungsnachweis** zeigt, wer ein Objekt nutzt: mit Programmart, bei Tabellen mit Zugriffsart lesend oder schreibend, mit Fundstellen samt Zeilennummer — und bei Strukturen mit den Feldern, die an der jeweiligen Stelle angefasst werden. Zu einer Tabelle erscheinen auch die Views und CDS Views, die auf ihr aufsetzen.

> **Aufbau ist nie begrenzt**
> Die Mengengrenzen gelten für Verwendungen — nie für das, woraus ein Objekt besteht. Die Datenelemente einer Tabelle werden immer vollständig gesammelt, sonst ließe sie sich im Zielsystem nicht aktivieren.

SAP-Standard ausschließen

Maßstab ist die Auslieferungseinheit des Pakets — dieselbe Linie, die SAP selbst zieht. Sie trifft anders als eine Namensprüfung auf Z und Y auch Kundenobjekte in reservierten Namensräumen und SAP-Objekte, die mit Z beginnen.

7 Prüfstandards dokumentieren: ZDOCUMIZER_ATC

Die dritte Sicht: nicht was im System steht, sondern woran es gemessen wird.

Modus	Beantwortet
Prüfvariante (V)	Was **wird** geprüft — eine Variante oder alle
Katalog (K)	Was **könnte** geprüft werden — alle aktivierten Checks

```
1       Prüfvariante     Eigentümer, transportierbar, versteckt
1.1       Kategorie
1.1.1       Check        Meldungen mit Code, Priorität, Kundenpriorität,
                         Pseudokommentar, Pragma, Ausnahmen, Text
```

Die Checkliste einer Variante steht nicht lesbar in der Datenbank; der Report liest sie über die Code-Inspector-API. Ein Check, den eine Variante nennt, den es aber nach einem Upgrade nicht mehr gibt, bekommt trotzdem ein Kapitel — mit dem Vermerk *nicht ladbar*.

> **Gemessen**
> Standardvariante `DEFAULT`: **17 Checks, 1.198 Meldungen**, Roundtrip identisch.

8 Ausgabe: XML und HTML

Beide Formate entstehen aus demselben Kapitelmodell im selben Lauf.

XML — vollständig, versioniert, wieder einlesbar

```
<documizer version="1.1" type="DEV">
  <meta>
    <system>DV1</system> <client>100</client> <language>D</language>
    <truncated>false</truncated> <node_limit>2000</node_limit> <node_total>44</node_total>
  </meta>
  <chapters>
    <chapter id="1.1.1" level="3" type="OBJECT">
      <title>Z_TABLE - Kopfdaten</title>
      <object type="TABL" note="Feldliste"> … </object>
      <source> … </source>
      <definition kind="TABL"> … </definition>
    </chapter>
  </chapters>
</documizer>
```

- **Nie unbemerkt unvollständig.** Jede Kürzung — Objektgrenze, Zeilengrenze, Quelltextgrenze — steht im Kopf und am Datenblock.
- **Vergleichbar zwischen Systemen.** Benutzer, Zeitstempel und Mandant stehen nur im Kopf, nie in den Kapiteln.
- **Roundtrip-geprüft.** Auf Wunsch serialisiert der Report, liest wieder ein, serialisiert erneut und vergleicht Zeichen für Zeichen.
- **Erweiterbar.** Neue Inhalte kommen in neue Elemente; ältere Dokumente bleiben lesbar.

HTML — lesbar, heute und in Jahren

Eine einzige Datei mit Inhaltsverzeichnis, nummerierten Kapiteln, Tabellen und Quelltextblöcken; CSS eingebettet, kein JavaScript. SAP-Dokumentation wird von ITF in lesbaren Text mit Zwischenüberschriften umgesetzt. Lange Blöcke stehen eingeklappt hinter einer Zeile mit Bezeichnung und Umfang.

Ablage und Dateiname

Ziel	Verfügbar	Hinweis
Download auf PC	Dialog	Binär in UTF-8 — Umlaute bleiben erhalten
Ablage auf dem Applikationsserver	Dialog und Hintergrund	Für große Läufe im Hintergrund

Der Dateiname wird aus System, Startobjekt, Datum und Uhrzeit vorgeschlagen — etwa `documizer_dev_DV1_z_package1_20260922_101500` — und lässt sich frei überschreiben.

9 Import: Entwicklungen in ein anderes System bringen

Ein in System A erzeugtes XML einlesen und die darin definierten Objekte in System B anlegen — mit demselben Report, ohne Transport und ohne abapGit.

Was angelegt wird

Objektart	Angelegt
Domäne, Datenelement	mit Festwerten und Texten
Tabelle, Struktur, Sekundärindex	mit technischen Einstellungen und Fremdschlüsseln; Indizes nach der Tabelle
View, CDS-Quelle	mit Basistabellen und Join-Bedingungen; SQL-View entsteht beim Aktivieren
Report, Include	mit Textelementen und Dokumentation **in allen Sprachen**, GUI-Status
Klasse, Interface	als quelltextbasierte Fassung, wie SE24 und ADT sie zeigen
Funktionsgruppe	mit Includes, Bausteinen und deren Schnittstelle
Nachrichtenklasse	alle Nachrichten in allen gepflegten Sprachen
Verhaltensdefinition, Service-Definition, Metadata Extension	als Quelltext
Service-Binding	mit Bindungsart und Services — bewusst unveröffentlicht
IDoc-Segmenttyp, IDoc-Basistyp	als echtes Segment, mit der ganzen Hierarchie
UI5-Anwendung	mit allen Dateien und Verzeichnisbaum über die UI5-Repository-API
Adobe Form und Schnittstelle	aus SAPs eigener Serialisierung wiederhergestellt — derselbe Weg, den das Kopieren im Form Builder nimmt
Dynpro, Pflegedialog, Transaktion, View-Cluster	der komplette SM30-Apparat, auch Parametertransaktionen

Sicher von Anfang an

- **Testlauf ist vorbelegt.** Jeder Schritt meldet, was er täte — geschrieben wird erst, wenn Sie den Haken bewusst herausnehmen.
- **Bestehendes bleibt.** Ein Objekt, das es schon gibt, wird nur mit *Bestehende Objekte überschreiben* angefasst.
- **Die richtige Reihenfolge.** Domänen vor Datenelementen vor Tabellen vor Views vor Programmen — der Report sortiert selbst.
- **Gezielt aktiviert.** Aus den inaktiven Objekten des Benutzers nimmt der Import nur die eigenen; angefangene Arbeit anderer bleibt unberührt.
- **Geprüft nach dem Lauf.** Aktive Fassung, Syntax-Check für Programme — gemeldet wird, was auffällt.

Sie wählen aus, was kommt

Vor dem Import lässt sich eine Liste dazwischenschalten: eine Zeile je Objekt des Dokuments, mit **Zielname**, **Zielpaket** und einer Ankreuzspalte **Übergehen**.

- **Umbenennen** wirkt auf das ganze Dokument, nicht nur auf den Objektnamen: der neue Name wird wortgenau und unabhängig von Groß- und Kleinschreibung auch im Quelltext ersetzt. Die Wortgrenze verhindert Teiltreffer — `ZKUNDE` lässt `ZKUNDE2` in Ruhe. So lässt sich auch ein SAP-Objekt unter einem Z-Namen hereinholen, ohne das Original zu überschreiben.
- **Übergehen** nimmt die Zeile ganz aus dem Lauf: weder angelegt noch geändert, auch nicht nachgeprüft. Jede Abwahl steht vorweg im Kopf und noch einmal im Protokoll — ein Weglassen ohne Vermerk wäre eine stillschweigende Lücke.

Abhängigkeiten werden dabei nicht erzwungen: wer eine Domäne abwählt, deren Datenelement importiert wird, bekommt den bekannten Aktivierungsfehler. Der Report warnt davor, verbietet es aber nicht — oft ist das abgewählte Objekt im Zielsystem längst vorhanden und soll gerade nicht angefasst werden.

Der Testlauf zeigt, was sich ändern würde

Ein Testlauf, der nur *würde geändert* sagt, verschweigt das Wesentliche. Zu jedem Objekt, das es im Zielsystem schon gibt, stellt er deshalb den Quelltext gegenüber:

```
CLAS  ZCL_MAILER   Quelltext anders   +12 / -3 Zeilen
        12   DATA lv_a TYPE i.
        13 - lv_a = 1.
        13 + DATA lv_b TYPE i.
        14 + lv_a = 2.
        15   WRITE lv_a.
       ... 240 Zeilen unveraendert
```

Neue Zeilen grün, entfallende rot, zwei Zeilen Umgebung, unveränderte Strecken zusammengefasst. Den alten Stand liest der Report aus dem Zielsystem in derselben Zusammenstellung, die der Export erzeugt — sonst wären Unterschiede zu sehen, die keine sind. Ist nichts anders, steht *Quelltext gleich* mit der Zeilenzahl.

Pakete, Auftrag und Aufgabe

Vier Regelzeilen **alt** → **neu** lenken Objekte aus dem Quellpaket in ein Zielpaket — etwa `$TMP` → `Z_PACKAGE1`. Der Auftrag hängt am Paket jedes einzelnen Objekts. Fehlt unter dem Auftrag eine offene Aufgabe für Sie, legt der Report sie dialogfrei an; ohne diese Aufgabe würde das Korrektursystem den Import mit einer irreführenden Meldung ablehnen.

Ein Protokoll, das Ursachen nennt

Je Objekt steht, was geschehen ist und warum — mit dem echten Rückgabecode und der Meldung des SAP-Bausteins. Scheitert eine Aktivierung, nennt das Protokoll das fehlende Objekt dahinter:

```
PROG Z_REPORT_COPY   angelegt               560 Zeilen
TEXT Z_REPORT_COPY   Textelemente gesetzt   D:5 E:8
DOKU Z_REPORT_COPY   Dokumentation gesetzt  Sprachen D
DTEL Z_ACTION        DDIF_DTEL_ACTIVATE rc 8 fehlt oder inaktiv: Domäne Z_ACTION_D
```

Gemessen: eine RAP-Anwendung als eine Datei

Paketlauf mit Quelltext und Definitionsdaten: **34 Definitionen** — 16 CDS-Views, 8 Klassen, 2 Verhaltensdefinitionen, Service-Definition und -Binding, Tabellen, Domäne, Datenelement und 2 UI5-Anwendungen mit **76 Dateien in 12 Ordnern**. Roundtrip identisch.

10 Sicherheit und Betrieb

Berechtigungen bleiben SAPs Sache

Der IMG-Report prüft `S_TABU_NAM` vor jedem Tabellenzugriff; fehlt die Berechtigung, steht das im Dokument. Beim Import prüfen die gerufenen SAP-Bausteine `S_DEVELOP` gegen Objekt und Paket — und diese Prüfung bleibt eingeschaltet. Der Documizer ist damit kein Werkzeug, mit dem jemand Objekte anlegt, die er von Hand nicht anlegen dürfte.

Laufzeiten

Lauf	Umfang	Dauer
IMG, Projektmodus	351 Kapitel	0,48 s
DEV, Pakethierarchie	10 Pakete / 352 Objekte	0,23 s
DEV, CDS mit Quelltext	4 Objekte	0,08 s
DEV, Transportauftrag	75 Objekte aus 2 Aufgaben	wenige Sekunden

Die teuersten Anteile sind die Dokumentationstexte je Objekt und die Suche nach Verwendungen; beides ist per Kennzeichen steuerbar. Verweistiefe und Verwendungstiefe sind die wirksamsten Stellschrauben — bei einem großen Paket lohnt es, sie zurückzunehmen. Für Vollläufe: Mengengrenzen auf 0, Hintergrundjob, Ablage auf dem Applikationsserver.

11 Grenzen und Ausblick

Was der Documizer nicht originalgetreu anlegen kann, sagt er — statt es halb zu tun.

Thema	Stand
Customizing-Inhalte	bewusst nicht direkt geschrieben — dafür BC-Sets
Smart Forms	werden dokumentiert, nicht angelegt; Adobe Forms dagegen beides
Umgebung einer Fiori-App	ICF-Knoten, MIME-Objekte und Gateway-Registrierung werden dokumentiert, nicht angelegt
Berechtigungsvorschlagswerte	dokumentiert, nicht übertragen (SU22/SU24)
IDoc-Erweiterungstypen	noch nicht unterstützt; Basistypen und Segmente vollständig
Bestehende Dynpros	werden nicht überschrieben — SAP bietet dafür keinen dialogfreien Weg
Fehlende Pakete	werden nicht automatisch angelegt; das bleibt eine Entscheidung des Entwicklers
Vergleich im Testlauf	vergleicht Quelltext; für DDIC-Objekte ist ein Feldvergleich vorgesehen
Gelöschte Objekte beim Sender	erkennbar, sobald der Startpunkt im Dokumentkopf steht — vorbereitet, nicht umgesetzt
Weitere Formate	Markdown und Anzeige einer XML-Datei ohne Systemzugriff sind vorbereitet

12 Bezug, Lizenz und Mitwirkung

Der Documizer ist quelloffen. Die aktuelle Fassung der drei Reports und der vollständigen technischen Spezifikation liegt öffentlich auf GitHub.

> **Quelle**
> github.com/csbg-de/documizer

Dort finden Sie die Quelltexte der Reports `ZDOCUMIZER_IMG`, `ZDOCUMIZER_DEV` und `ZDOCUMIZER_ATC`, den Hilfsreport für die Reportdokumentation, die Spezifikation mit allen Entwurfsentscheidungen und diese Produktdokumentation. Beiträge, Fehlermeldungen und Erfahrungsberichte aus anderen Systemständen sind willkommen.

Lizenz

Punkt	Regelung
Lizenz	Apache License 2.0
Copyright	2026 C. Gathmann Computer-System-Beratung GmbH (CSBG), Bremen
Nutzung	Weitergabe und Änderung erlaubt, auch kommerziell
Bedingung	Der Copyright- und Lizenzvermerk bleibt erhalten
Name	*consiness* und *consiness Documizer* sind keine lizenzierten Bestandteile — der Code darf übernommen werden, der Name nicht weitergeführt

Weil die Reports per Copy & Paste in ein fremdes System wandern und die Datei `LICENSE` dabei nicht mitkommt, steht der Vermerk im Kopfkommentar **jeder** Quelle. Beim Kopieren darf er nicht entfernt werden — er ist die einzige Stelle, an der die Herkunft im Zielsystem noch erkennbar ist.

> **Entstehung**
> Entworfen, spezifiziert und am laufenden System verifiziert von Christian Gathmann; der Code ist mit KI-Unterstützung entstanden. Jede Funktion dieses Handbuchs ist in einem echten SAP-System gelaufen, bevor sie hier beschrieben wurde.

13 Kontakt

Sie möchten den Documizer in Ihrem System einsetzen, eine Anwendung dokumentieren oder Entwicklungen übertragen?

Christian Gathmann
gathmann@csbg.de
C. Gathmann Computer-System-Beratung GmbH (CSBG), Bremen

github.com/csbg-de/documizer

